GPU-Accelerated Martingale Posterior Neural Processes for MNIST Image Completion

Jose Perales
EN605.617: Introduction to GPU Programming
Johns Hopkins University

Spring 2026

Abstract

The Martingale Posterior Neural Process (MPNP, Lee et al., ICLR 2023) replaces the arbitrary latent prior of a standard Neural Process with predictive resampling. At every training step the model draws synthetic context sets from its own predictive distribution and refits. The payoff is calibrated, prior-free uncertainty over functions. An MPNP provides calibrated doubt, not just inference, the kind a deployed model needs before it brakes for a pedestrian, flags a tumor, or sizes a tail-risk position. The cost is that every gradient step now runs many encoder and decoder forward passes per task, making the model GPU-bound.

This paper takes a working PyTorch MPNP implementation for MNIST image completion and uses it as a vehicle for a tour of the CUDA stack. It walks through which NVIDIA library kernel each MPNP op actually launches: cuBLAS `cublasGemmEx` GEMMs for the MLPs, native ATen softmax and LayerNorm kernels for the attention blocks, TensorIterator elementwise kernels for the loss, the in-tree Philox RNG, and the caching allocator. Pseudo-context resampling multiplies those launches by $K+2$ every training step, which is what makes MPNP GPU-bound in the first place.

1 Introduction

Models that return only a point estimate are often unsuitable in risky environments. Autonomous driving needs a confidence band on the pedestrian-detection score before it commits to braking. In clinical decision support, a model that flags its own uncertainty is more useful than one that is confidently wrong. Quantitative finance needs tail risk that is calibrated, not just estimated. Uncertainty quantification is critical wherever predictions inform consequential actions.

Lee, Yun, Nam, Fong, and Lee provide a solution to uncertainty quantification with the Martingale Posterior Neural Process (MPNP) [1], a Bayesian-style Neural Process that does not require a prior. Following Fong, Holmes, and Walker [2], they replace the prior with predictive resampling. At every step, the model generates K synthetic context sets from its own predictive distribution, refits, and uses the spread of refits as the posterior. Posterior updates form a martingale (expected belief over self-generated data does not drift). The result is coherent uncertainty without ever specifying a prior.

However, predictive resampling comes at a computational cost. The new objective requires pseudo-context forward passes through the encoder, decoder, and per-target Gaussian negative log-likelihood (NLL) per task per training step. This is a natural fit for the GPU, and it makes MPNP a useful case study in GPU programming. This paper traces how a PyTorch implementation

maps onto the CUDA stack underneath: which library each MPNP op dispatches to, and which call sites the pseudo-context loop multiplies.

2 Math Review

2.1 Martingale Posterior Distributions

Classical Bayesian inference asks "what do I believe about the parameter θ?"; a question that demands a prior $\pi(\theta)$ before any data is observed. Fong, Holmes, and Walker flip the question: "how do I expect future observations to look, given what data I already have?" That requires only a one-step predictive $P_n(\cdot \mid y_{1:n})$, no prior [2].

Uncertainty about θ becomes uncertainty about the missing future. Fong et al. design a predictive resampling mechanism to forward-simulate a long imaginary future from the current predictive, refit, and read off θ:

$$Y_i \sim P_{i-1}, \quad i = n{+}1, \ldots, N \quad \Longrightarrow \quad \theta_N = \theta(P_N) \sim \Pi_N(\cdot \mid y_{1:n}). \tag{1}$$

Repeating the simulation many times yields a full posterior Π_N over θ. The predictive sequence $\{P_n\}$ is a martingale, $\mathbb{E}[P_{n+1} \mid P_n] = P_n$, so beliefs are consistent and do not drift on self-generated data. There is no prior to choose and no likelihood to specify, only a predictive model and the samples it generates.

2.2 Neural Processes

The two classical answers to this problem space (predictions with calibrated uncertainty) each have a flaw. Neural networks are flexible, scalable function approximators but return only point predictions with no principled uncertainty. Gaussian processes give exact posterior mean and variance per query but cost $\mathcal{O}(n^3)$ per task and do not amortize across datasets.

Neural Processes (NPs) [3] sit in the middle. Like a Gaussian Process, an NP learns a distribution over functions from a context set $\mathcal{C} = \{(x_i, y_i)\}_{i=1}^n$, but it does so with a neural network and amortizes inference into a single forward pass:

$$p(y_t \mid x_t, \mathcal{C}) = \int p(y_t \mid x_t, z)\, q(z \mid \mathcal{C})\, dz. \tag{2}$$

A permutation-invariant encoder $q_\phi(z \mid \mathcal{C})$ (MLP plus mean-pool) maps the context set to a latent distribution, and a decoder $p_\theta(y_t \mid x_t, z)$ predicts each target from the latent. The model is trained by maximizing an evidence lower bound. The remaining problem is the same one Bayesian inference always has. The latent prior $p(z)$ is arbitrary, so the posterior is rarely calibrated.

2.3 Martingale Posterior Neural Processes

Lee et al. plug Fong's predictive resampling inside the NP training loop [1]. The model uses the same encoder and decoder, plus a pseudo-context generator, an Induced Set Attention Block (ISAB) [4], that samples K synthetic context sets $\mathcal{Z}_0^{(k)}$ from the model's own current predictive. This is a neural analogue of predictive resampling.

The training objective combines three terms:

$$\boxed{\mathcal{L}_{\text{MPNP}} = \underbrace{\mathcal{L}_{\text{amort}}}_{\text{predict from real } \mathcal{C}} + \underbrace{\mathcal{L}_{\text{marg}}}_{\substack{\text{log-mean-exp over } K \\ \text{pseudo-augmentations}}} + 0.1 \cdot \underbrace{\mathcal{L}_{\text{pseudo}}}_{\substack{\text{pseudo}\to\text{target} \\ \text{consistency}}}.} \tag{3}$$

Here

$$\mathcal{L}_{\text{amort}} = -\mathbb{E}_{q(z|\mathcal{C})}\left[\log p(y_T \mid x_T, z)\right], \tag{4}$$

$$\mathcal{L}_{\text{marg}} = -\log\left(\frac{1}{K}\sum_{k=1}^{K} p(y_T \mid x_T, \theta_k)\right), \qquad \theta_k = \theta\left(\mathcal{C} \cup \mathcal{Z}_0^{(k)}\right), \tag{5}$$

and $\mathcal{L}_{\text{pseudo}}$ scores the predictive quality of the pseudo-set alone.

$\mathcal{L}_{\text{marg}}$ requires K pseudo-augmented forward passes per task per step. With $B = 64$, $T = 784$, hidden width $h = 256$, and $K = 5$, the inner forward graph holds $\mathcal{O}(B \cdot K \cdot T \cdot h)$ activations and launches at least $K+1$ separate sequences of linear-layer, decode, and log-prob kernels per step. This is the bottleneck that the rest of the paper targets.

3 MNIST Example

3.1 Task Formulation

Each 28×28 MNIST image is flattened into 784 coordinate-intensity pairs:

- $x_i \in [-1, 1]^2$: normalized pixel coordinate (row, column).
- $y_i \in [0, 1]$: pixel intensity.

Per training task, a random subset of $n \in [10, 300]$ pixels is revealed as context. The model must predict all 784 pixel intensities and their per-pixel standard deviations. MNIST is a fitting test bed for a Neural Process because it has variable-size context, genuine spatial structure, and uncertainty that should localize on unobserved pixels and ambiguous edges.

3.2 Architecture and Hyperparameters

The MPNP implementation reuses the encoder/decoder MLPs from `neural_process.py`, adapted from Dupont's NP repository [5]. The pseudo-context generator is a single ISAB block built from two `MAB` (multihead-attention block) layers with 8 heads.

The shapes that appear throughout Section 4 are listed in Table 1.

Tensor	Shape	Notes
context (x, y)	$(B, n, 3)$	n varies per batch
target x_T	$(B, 784, 2)$	fixed
target y_T	$(B, 784, 1)$	fixed
pseudo-context	$(B, K, M, 3)$	K=5, M=30
latent z	(B, K, z_{dim})	z_{dim}=256

Table 1: Tensor shapes used by the MPNP training step.

Hyperparameters from `train_inpainting.py` are listed in Table 2.

Hyperparameter	Value
Hidden width $h = r = z$	256
Pseudo-points M	30
Pseudo-samples K	5
Loss weights	$(\mathcal{L}_{\text{amort}}, \mathcal{L}_{\text{marg}}, \mathcal{L}_{\text{pseudo}}) = (1.0, 1.0, 0.1)$
Optimizer	AdamW, lr $5{\times}10^{-4}$, weight decay 10^{-5}
LR schedule	Cosine, $T_{\max} = 150$
Batch size	64
Epochs	150
Gradient clip	$\|g\|_2 \leq 5$
Train / val / test split	50 000 / 10 000 / 10 000

Table 2: Training hyperparameters.

3.3 Qualitative Results

After 150 epochs the model reaches a test NLL of -1.941 (best validation NLL -1.947 at epoch 145; approximately 100.66 s/epoch over approximately 4.19 h on a single GPU). Lower is better, and for a Gaussian over $[0, 1]$ pixels this corresponds to σ in the low tens of percent on average.

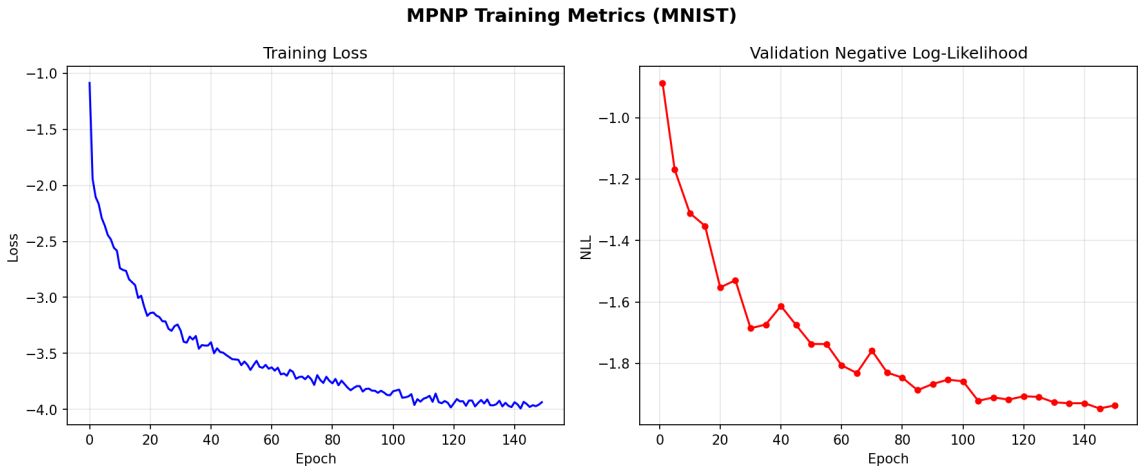

Figure 1: Training loss and validation NLL across 150 epochs. Loss decreases smoothly; validation NLL tracks training loss with no prior-collapse pathology and no overfitting.

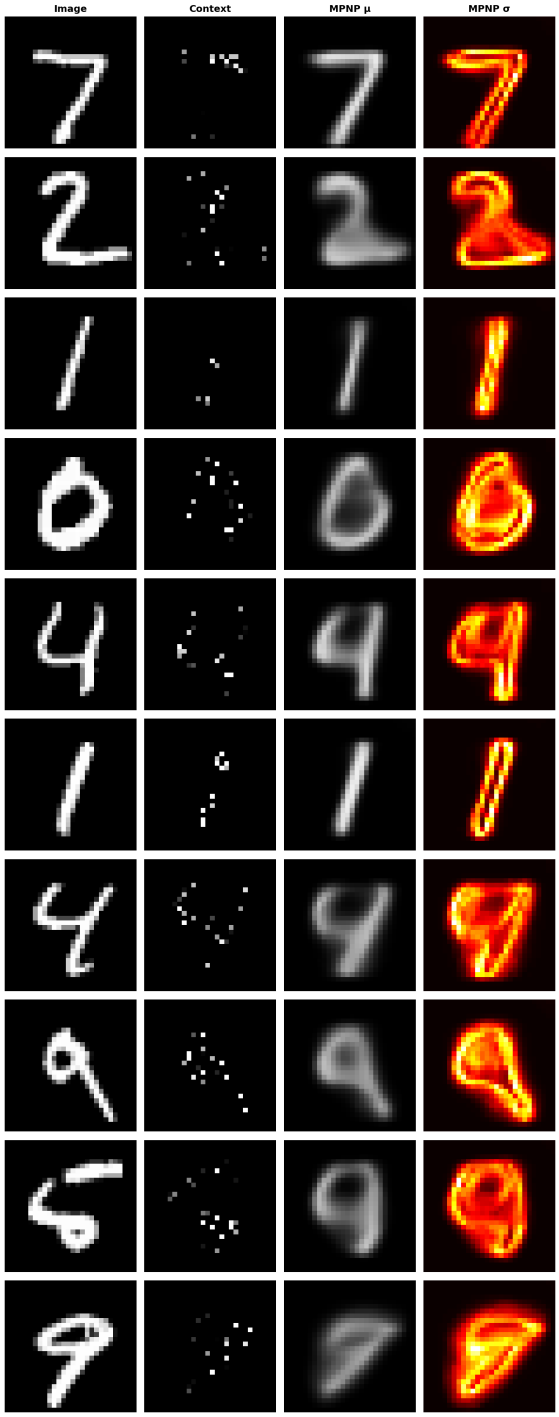

Figure 2: Test-set image completion. Columns: original digit, observed context pixels, predicted mean μ, predicted standard deviation σ. Uncertainty concentrates on unobserved regions and on ambiguous strokes, matching the calibration behavior reported by the MPNP paper.

The remainder of the paper analyzes how this training loop runs on the GPU.

4 CUDA Programming inside PyTorch

The MPNP implementation in `train_inpainting.py` and `martingale_posterior_neural_process.py` abstracts over all the CUDA programming covered in this course, yet a single training step issues dozens of CUDA kernel launches across cuBLAS, the caching allocator, and ATen's library of elementwise and reduction kernels. This section inventories the GPU-touching call sites and names the backend behind each one.

4.1 The PyTorch, ATen, CUDA dispatch stack

Before naming individual kernels it helps to fix the layering, because the project itself contains no CUDA C++ code. Every tensor operation flows through three layers [7, 13]:

1. **Python / `torch.nn`.** The user-facing API (`nn.Linear`, `Normal.log_prob`, `loss.backward()`). Each call constructs an op and hands it to ATen.

2. **ATen** (*A Tensor library*). PyTorch's C++ op library and dispatcher [12, 13]. For each op (`aten::addmm`, `aten::mean`, `aten::log`, ...) ATen looks at the input tensors' device and dtype and routes the call to a backend implementation. The `aten::` prefix visible in profiler output is just this namespace.

3. **CUDA backend.** The actual GPU code. For CUDA tensors, ATen routes either to a vendor library shipped by NVIDIA (cuBLAS for GEMM [8], cuDNN for convolutions and some normalizations [11]) or to one of ATen's own native CUDA kernels for ops NVIDIA does not provide, such as the `TensorIterator` elementwise and reduction kernels [7] behind `add`, `mul`, `log`, `mean`, and so on, dedicated kernels for `softmax` and `layer_norm`, and the in-tree Philox-4x32 random-number generator [6].

The practical consequence is that every kernel inventoried in the rest of this section comes from one of those two sources — a vendor library or an ATen native CUDA kernel — and reasoning about cost means knowing which.

4.2 Where the GPU runs in this project

Almost every line of `train_inpainting.py` and `martingale_posterior_neural_process.py` exercises the GPU, in three distinct phases: the data path, the forward path, and the loss-and-backward path.

Data path. The MNIST point cloud is built on the CPU. `MNISTPointCloud.__init__` precomputes a 784×2 coordinate grid as a CPU tensor; `__getitem__` returns a CPU `(coords, values)` pair. The DataLoader is constructed with `pin_memory=True`, which routes those CPU batches through page-locked host memory so the subsequent `x_full.to(device)` in `train_epoch` can issue an asynchronous `cudaMemcpyAsync` instead of a blocking copy [6]. The synthetic `torch.randperm(n, device=x.device)` call inside `context_target_split` runs the permutation on the GPU using the in-tree Philox-4x32 generator that backs PyTorch's CUDA RNG [10].

Forward path. A single training step calls `model.compute_loss(x_ctx,y_ctx,x_tgt,y_tgt)`, which in turn invokes `encode_set` once on the real context, then K=5 more times on each combined context, plus `decoder` calls in matching number. Each of those calls is a stack of `nn.Linear`

layers separated by `ReLU` activations; each `nn.Linear` dispatches to `aten::addmm`, which is a cuBLAS GEMM with bias fusion [8]. The mean-pool aggregator inside `encode_set` is a single `aten::mean` reduction. The hand-written `MultiHeadAttention` block inside the `ISAB` pseudo-context generator issues two `aten::matmul` kernels, an `aten::softmax`, and several small reshapes. The reparameterization $z = \mu + \sigma \odot \epsilon$ inside `q_context.rsample()` is one elementwise kernel scheduled through `TensorIterator` [7].

Loss and backward path. `Normal.log_prob(y_tgt)` expands into a sequence of elementwise kernels (`log`, `pow`, `sub`, `div`) followed by an `aten::mean` reduction; this fires K+1 times per step. The marginal-likelihood term in `compute_loss` stacks K such log-likelihoods and reduces them with a hand-rolled log-sum-exp (`max` → `sub` → `exp` → `mean` → `log`, four kernels where `torch.logsumexp` would be one). Calling `loss.backward()` traverses the autograd graph in reverse and launches the gradient kernel for each forward op above on the same CUDA stream the forward used [6]. The `optimizer.step()` call on `torch.optim.AdamW` fires a fused multi-tensor elementwise kernel that updates every parameter in one launch.

Table 3 summarizes the mapping. The shapes use the project's actual hyperparameters: batch size B=64, hidden width h=256, context size $n \in [10, 300]$, target size T=784 pixels per image, latent width z=256, M=30 pseudo-context points, 8 attention heads, and K=5 pseudo-context samples.

MPNP step	ATen op	Backend / kernel	Tensor shape
Encoder MLP (`Linear` + `ReLU`)	`addmm`	cuBLAS `cublasGemmEx` [8]	$(B{\cdot}n, 256)$ $\times(256, 256)$
Mean-pool over context	`mean`	ATen reduction, `__shfl_down_sync` [9]	$(B, n, 256)$ $\rightarrow(B, 256)$
ISAB MAB stack ($\times K$)	`matmul`, `softmax`, `layer_norm`	cuBLAS batched GEMM + ATen CUDA kernels [8]	$(B, 8, 30, 32)$
Reparam. $z{=}\mu{+}\sigma\epsilon$	`add`, `mul`	TensorIterator + Philox-4x32 [7]	$(B, 256)$
Decoder MLP	`addmm`	cuBLAS `cublasGemmEx` [8]	$(B{\cdot}T, 256)$ $\times(256, 256)$
Per-pixel Gaussian NLL	`log`, `pow`, `sub`	TensorIterator elementwise [7]	$(B, 784, 1)$
Manual log-sum-exp over K	`max`, `exp`, `log`	5 ATen kernels in sequence	(K, B)
AdamW step	`_foreach_*`	multi-tensor elementwise	all params

Table 3: Mapping from MPNP operations to PyTorch ops, CUDA backend kernels, and tensor shapes.

Two structural facts about this table matter. First, the encoder, decoder, and per-pixel log-likelihood each fire K+2 times per training step (once for the real context pass and K more times inside the pseudo-context loop) so any per-launch cost is multiplied accordingly. Second, the only ops whose tensor shapes vary batch-to-batch are the encoder GEMM and the mean-pool reduction (both because n varies); every other shape is fixed by the architecture.

5 Conclusion

After 150 epochs the model reaches a test NLL of -1.941 with stable training, and image completions (Figure 2) reproduce the qualitative behavior of the original MPNP paper: predicted σ concentrates on unobserved regions and ambiguous edges, the calibrated-uncertainty pattern that motivates the method.

The exercise also makes the implicit CUDA stack underneath PyTorch concrete. Every `nn.Linear` in the encoder and decoder is a cuBLAS GEMM, every `Normal.log_prob` is a chain of TensorIterator elementwise kernels, and every `torch.randperm` draw runs through the in-tree Philox generator. Pseudo-context resampling multiplies all of those launches by $K+2$ per step, which is what makes MPNP a useful vehicle for the tour rather than just another MNIST model.

References

[1] H. Lee, E. Yun, G. Nam, E. Fong, and J. Lee. Martingale Posterior Neural Processes. *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2304.09431.

[2] E. Fong, C. Holmes, and S. G. Walker. Martingale Posterior Distributions. *Journal of the Royal Statistical Society: Series B*, 2023. https://arxiv.org/abs/2103.15671.

[3] M. Garnelo, J. Schwarz, D. Rosenbaum, F. Viola, D. J. Rezende, S. M. A. Eslami, and Y. W. Teh. Neural Processes. *ICML Workshop*, 2018. https://arxiv.org/abs/1807.01622.

[4] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh. Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. *International Conference on Machine Learning*, 2019. https://arxiv.org/abs/1810.00825.

[5] E. Dupont. Neural Processes (PyTorch). https://github.com/EmilienDupont/neural-processes.

[6] PyTorch documentation. CUDA semantics notes. https://docs.pytorch.org/docs/stable/notes/cuda.html.

[7] E. Z. Yang. PyTorch internals. 2019. https://blog.ezyang.com/2019/05/pytorch-internals/.

[8] NVIDIA Corporation. cuBLAS Library User Guide. https://docs.nvidia.com/cuda/cublas/.

[9] K. Patel and S. Jones. Cooperative Groups: Flexible CUDA Thread Programming. NVIDIA Developer Blog, 2017. https://developer.nvidia.com/blog/cooperative-groups/.

[10] NVIDIA Corporation. cuRAND Library User Guide. https://docs.nvidia.com/cuda/curand/.

[11] NVIDIA Corporation. cuDNN Developer Guide. https://docs.nvidia.com/deeplearning/cudnn/.

[12] PyTorch documentation. PyTorch C++ API (ATen). https://pytorch.org/cppdocs/.

[13] PyTorch documentation. Registering a Dispatched Operator in C++. https://pytorch.org/tutorials/advanced/dispatcher.html.